

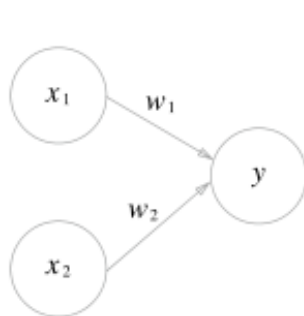
Neural Networks

Prof. Tae-Hyoung Park

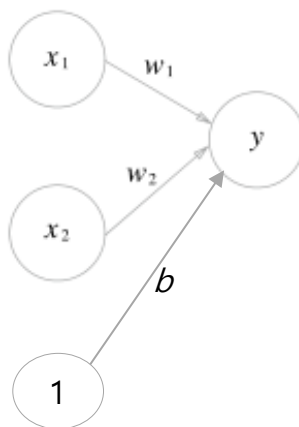
Dept. of Intelligent Systems & Robotics, CBNU

Perceptron

- Perceptron (F.Rosenblatt, 1975)
 - 신경망의 기원



$$y = \begin{cases} 0 & (w_1x_1 + w_2x_2 \leq \theta) \\ 1 & (w_1x_1 + w_2x_2 > \theta) \end{cases}$$



x_1, x_2 : input, y : output
 w_1, w_2 : weight, b : bias,

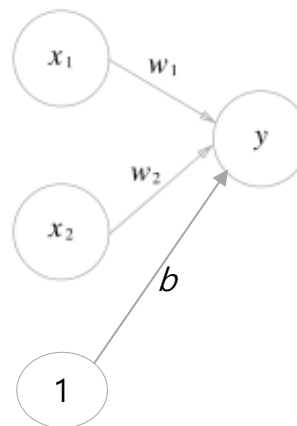
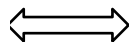
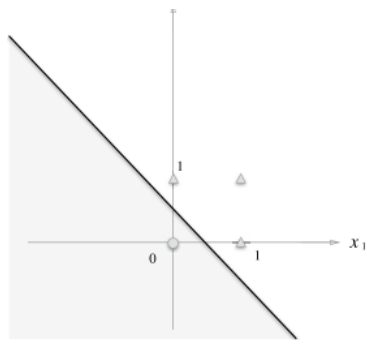
$$y = \begin{cases} 0 & (b + w_1x_1 + w_2x_2 \leq 0) \\ 1 & (b + w_1x_1 + w_2x_2 > 0) \end{cases}$$

- Single-layer perceptron => Linear classification 가능
Non-linear classification 불가

Perceptron

(ex) OR gate

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	1

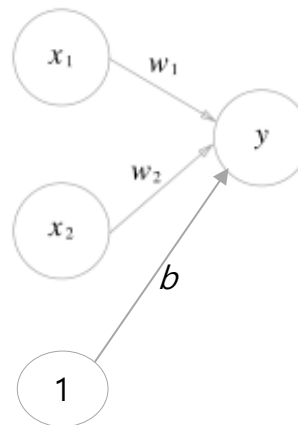
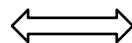
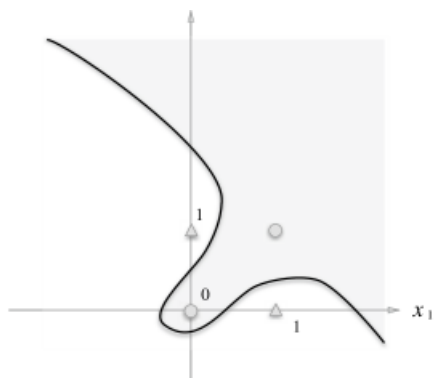


$$w_1 = 1.0, w_2 = 1.0, b = -0.5$$

$$y = \begin{cases} 0 & (-0.5 + x_1 + x_2 \leq 0) \\ 1 & (-0.5 + x_1 + x_2 > 0) \end{cases}$$

(ex) XOR gate

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

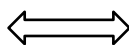


$$w_1 = ??, w_2 = ??, b = ??$$

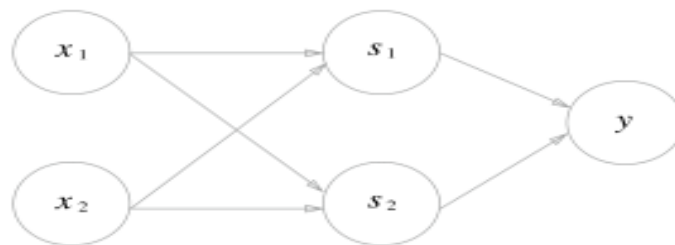
Multi-Layer Perceptron

- Multi-Layer Perceptron (MLP)
 - 1개 이상의 hidden layer
 - Nonlinear classification 가능

(ex) XOR gate – 2 layers



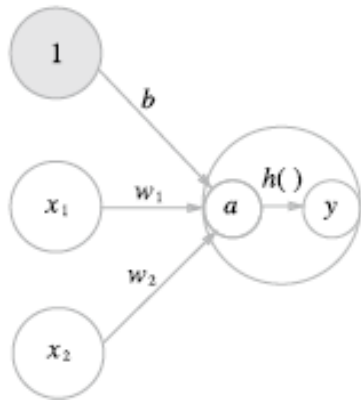
x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0



```
def XOR(x1, x2):  
    s1 = NAND(x1, x2)  
    s2 = OR(x1, x2)  
    y = AND(s1, s2)  
    return y
```

Multi-Layer Perceptron

- Activation function (활성화 함수): $h(x)$

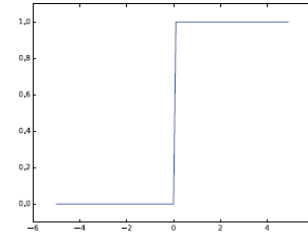


$$a = b + w_1x_1 + w_2x_2$$

$$y = h(a)$$

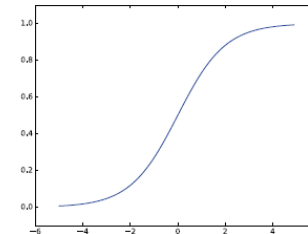
step

$$h(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases}$$



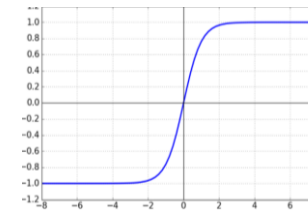
sigmoid

$$h(x) = \frac{1}{1 + \exp(-x)}$$



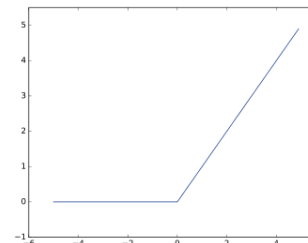
tanh

$$h(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



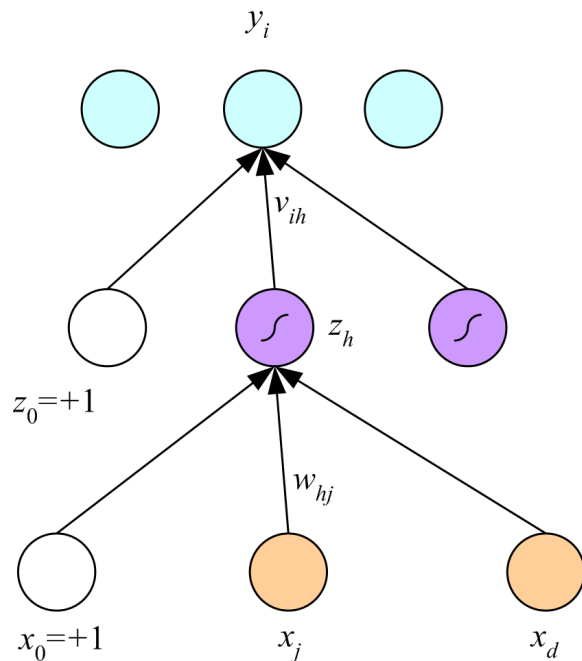
relu

$$h(x) = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases}$$



Multi-Layer Perceptron

- Multilayer Perceptron (Rumelhart, 1986)
 - Input layer (layer 0)
 - Hidden layers (layer 1, 2,)
 - Output layer

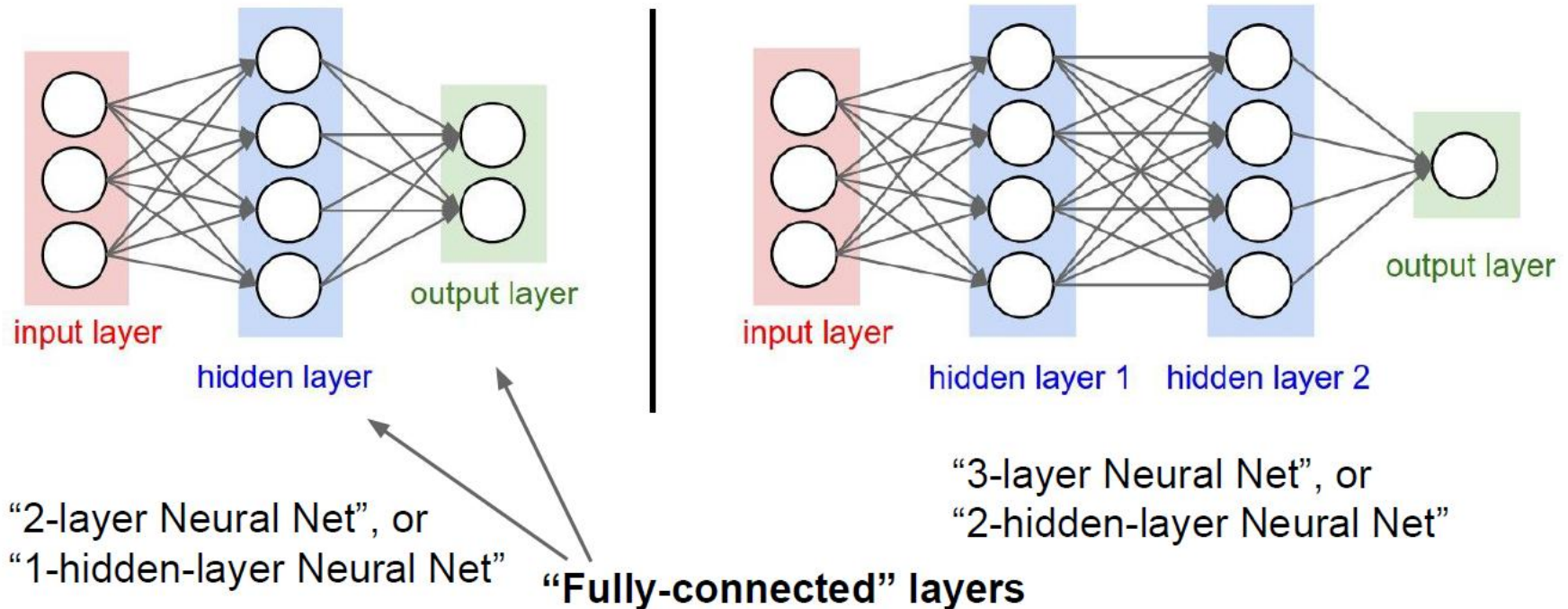


$$y_i = \mathbf{v}_i^T \mathbf{z} = \sum_{h=1}^H v_{ih} z_h + v_{i0}$$

$$z_h = \text{sigmoid}(\mathbf{w}_h^T \mathbf{x})$$
$$= \frac{1}{1 + \exp\left[-\left(\sum_{j=1}^d w_{hj} x_j + w_{h0}\right)\right]}$$

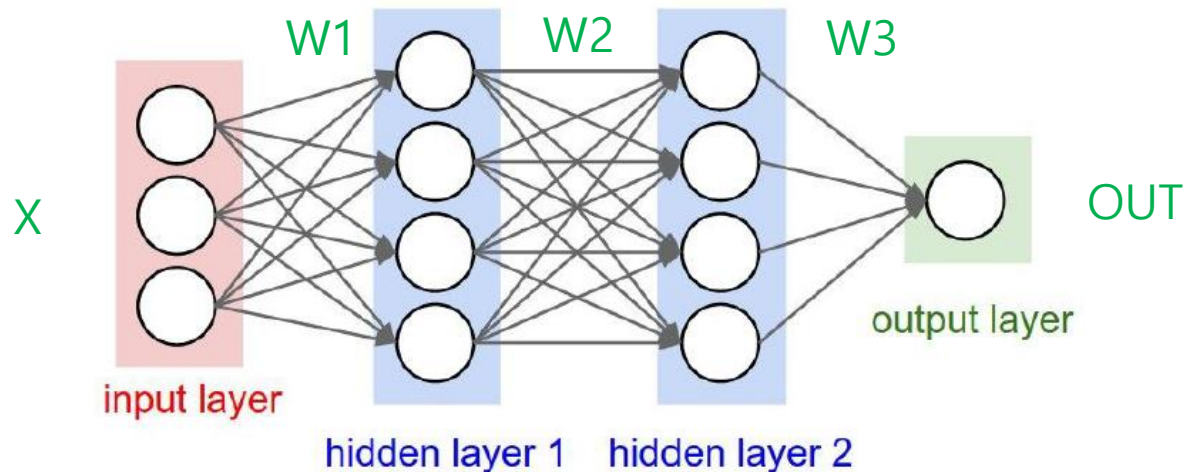
Multi-Layer Perceptron

- Architecture



Multi-Layer Perceptron

- Feedforward computation

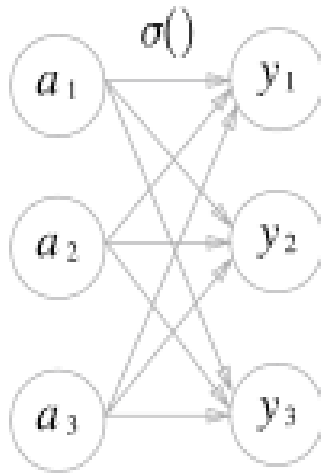


```
# forward-pass of a 3-layer neural network:  
f = lambda x: 1.0/(1.0 + np.exp(-x)) # activation function (use sigmoid)  
x = np.random.randn(3, 1) # random input vector of three numbers (3x1)  
h1 = f(np.dot(W1, x) + b1) # calculate first hidden layer activations (4x1)  
h2 = f(np.dot(W2, h1) + b2) # calculate second hidden layer activations (4x1)  
out = np.dot(W3, h2) + b3 # output neuron (1x1)
```

$$W1 = \begin{bmatrix} w_{11}^1 & \cdots & w_{14}^1 \\ \vdots & \ddots & \vdots \\ w_{31}^1 & \cdots & w_{34}^1 \end{bmatrix} \quad W2 = \begin{bmatrix} w_{11}^2 & \cdots & w_{14}^2 \\ \vdots & \ddots & \vdots \\ w_{41}^2 & \cdots & w_{44}^2 \end{bmatrix} \quad W3 = \begin{bmatrix} w_1^3 \\ \vdots \\ w_4^3 \end{bmatrix}$$

Multi-Layer Perceptron

- Softmax function
 - Classification network 의 출력



$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)}$$

Multi-Layer Perceptron

- Loss Function

- 1) Mean square error (MSE)

$$E = \frac{1}{2} \sum_k (y_k - t_k)^2$$

y_k : 신경망 출력, t_k : 정답 레이블

=> Regression (회귀분석) 문제에 많이 활용

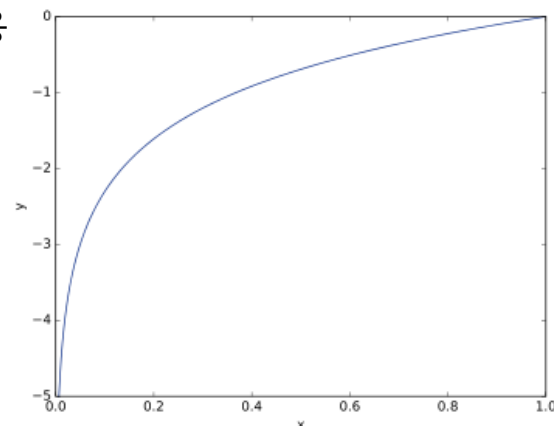
- 2) Cross entropy error (CEE)

$$E = - \sum_k t_k \log y_k$$

=> Classification 문제에 많이 활용

* Mini-batch 학습

$$E = - \frac{1}{N} \sum_n \sum_k t_{nk} \log y_{nk}$$

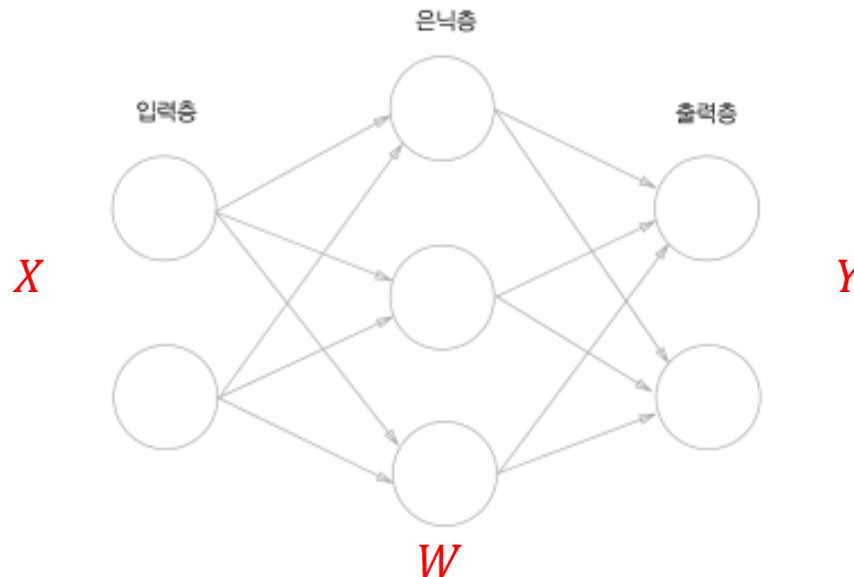


Training MLP

- Learning algorithm
 - Loss function 을 최소화 하는 W 를 구하는 문제
 - Optimization 문제

$$W^* = \arg \min \frac{1}{2} \sum_k (y_k - t_k)^2$$

$$\text{s.t.} \quad y_k = f(X, W), \quad k = 1, \dots, N$$

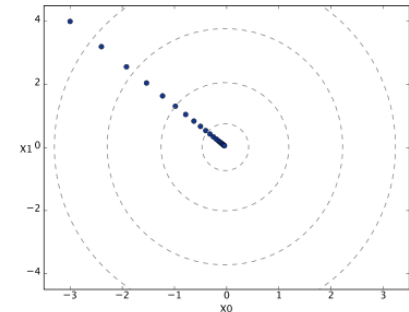
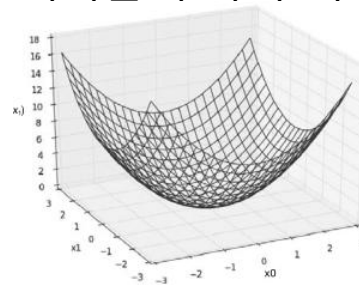


Training MLP

- Approach
 - Non-convex problem
 - Local search technique
- Gradient descent method (경사하강법)
 - Hill-climbing 방법
 - ① 초기 값을 임의로 설정하고
 - ② 현재 값 주변을 탐색하여, **주변에서 가장 좋은 값**을 선택. 개선이 없으면 종료 => 기울기 (gradient) 값 사용
 - ③ 선택된 값을 현재 값으로 하여 더 이상 개선이 없을 때 반복
 - 문제
 - 초기 값에 의존 => global 최적을 구하기 어려움

– 예 $f(x_0, x_1) = x_0^2 + x_1^2$

$$x_0 = x_0 - \eta \frac{\partial f}{\partial x_0}$$
$$x_1 = x_1 - \eta \frac{\partial f}{\partial x_1}$$



Training MLP

- Learning a Perceptron

- Error function

$$E = \frac{1}{2} (t - y)^2 = \frac{1}{2} (t - W^T X)^2$$

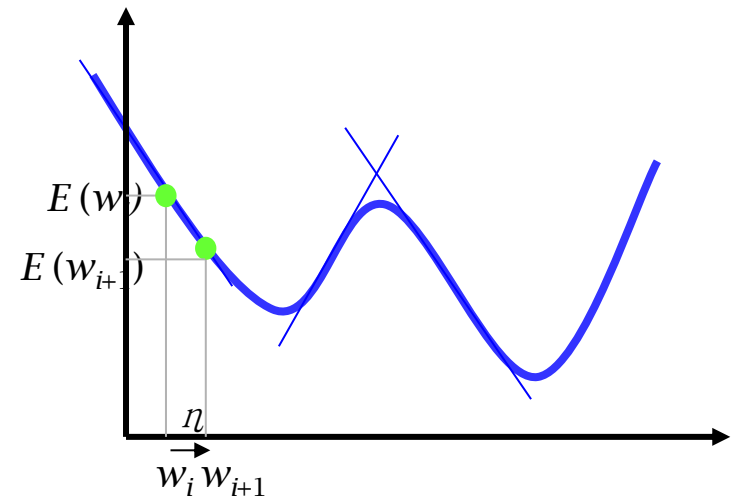
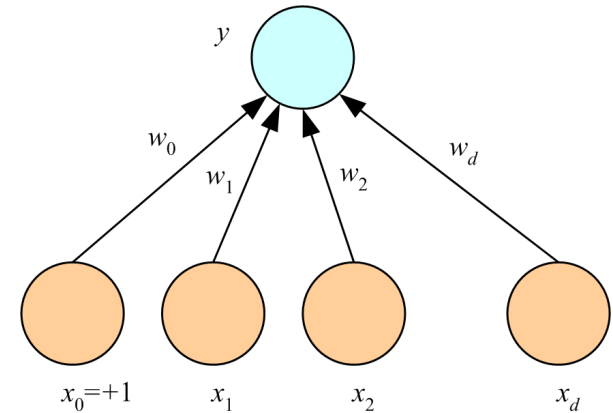
- $W^* = \arg \min E(W, X)$

- Update rule

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i}, \forall i \quad \eta: \text{ learning factor}$$
$$w_{i+1} = w_i + \Delta w_i$$

$$\Delta w_i = \eta (t - y) x_i, \quad i = 0, \dots, d$$

Update = LearningFactor · (DesiredOutput - ActualOutput) · Input



Gradient descent

Training MLP

(Ex)

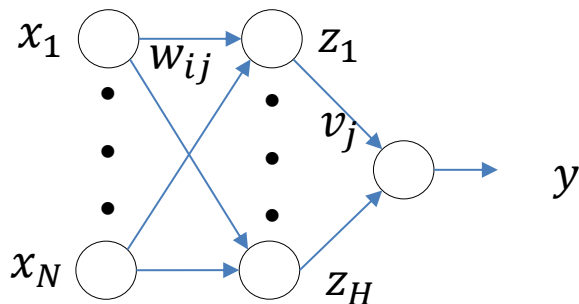
학습하는 과정에 있어서 현 단계의 연결 강도가 $[0.2 \ 0.5 \ 0.3]$ 인 경우, 목표치가 1인 학습 패턴 $X = [0 \ 0 \ 1]$ 을 신경망에 입력한 결과, -1 이 출력되었다. 이때의 연결 강도 변화량 ΔW 와 다음 학습 단계에서의 연결 강도 W 를 구하라. 단, 학습률은 1이고, 학습 신호는 목표치와 출력의 차이이다.

$$\Delta W = \eta(t - y) X = 1 \cdot (1 - (-1)) \cdot [0 \ 0 \ 1] = [0 \ 0 \ 2]$$

$$W_{k+1} = W_k + \Delta W = [0.2 \ 0.5 \ 0.3] + [0 \ 0 \ 2] = [0.2 \ 0.5 \ 2.3]$$

Training MLP

- Learning MLP



$$y_k = \sum_{j=0}^H v_j z_j$$

$$z_j = \text{sigmoid}(\mathbf{w}_j^T \mathbf{x})$$

$\mathbf{x}$

Forward

error

$$E = \frac{1}{2} (t - y)^2$$

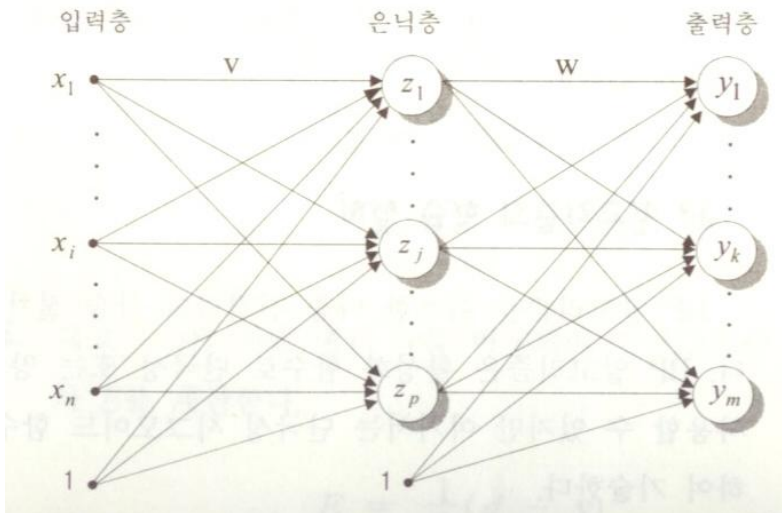
$$\Delta v_j = -\eta \frac{\partial E}{\partial v_j} = \eta (t - y) z_j$$

$$\begin{aligned} \Delta w_{ij} &= -\eta \frac{\partial E}{\partial w_{ij}} \\ &= -\eta \frac{\partial E}{\partial y} \frac{\partial y}{\partial z_j} \frac{\partial z_j}{\partial w_{ij}} \\ &= \eta (t - y) v_j z_j (1 - z_j) x_i \end{aligned}$$

Backward

Training MLP

- Error Back Propagation Algorithm
 - 오류 역전파 알고리즘
 - D.Parker, 1982



Step 1 : Initialize weights and counter

$V, W \leftarrow$ small random value
 $p \leftarrow$ number of training pattern pairs
 $k \leftarrow 1$
 $E \leftarrow 0$

Step 2 : Set learning rate α and $E_{\max}$

Step 3 : For each training pattern pair
 do Step 4-10 until $k = p$

Step 4 : Compute output of hidden layer

$$NET_z = XV^T$$

$$Z = \frac{1}{1 + e^{-NET_z}}$$

Step 5 : Compute output

$$NET_y = ZW^T$$

$$y = \frac{1}{1 + e^{-NET_y}}$$

Step 6 : Compute output error

$$E \leftarrow \frac{1}{2} (d - y)^2$$

Step 7 : Compute error signal of output layer

$$\delta_y = (d - y) y (1 - y)$$

Step 8 : Compute error signal of hidden layer

$$\delta_z = z(1 - z) \sum_{j=1}^m \delta_y w_{zj}$$

Step 9 : Update weights

$$W = W + \alpha \delta_y Z$$

$$V = V + \alpha \delta_z X$$

Step 10 : Increase counter and goto Step 3

$$k \leftarrow k + 1$$

Step 11 : Test stop condition

If $E < E_{\max}$ stop
 else, $E \leftarrow 0$ and goto Step 3

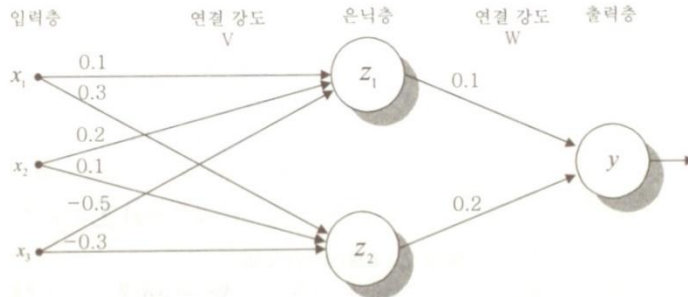
Training MLP

(Example)

그림과 같은 3계층 신경망을 BP 알고리즘으로 학습시켜 4개의 패턴을 2개의 클러스터로 분류하기 위해 초기 연결 강도 V 와 W 를 아래와 같이 설정하였다. 목표치가 -1 인 학습 패턴 A를 입력할 경우의 오차는 얼마인가? 단, 단극성 시그모이드 함수를 활성화 함수로 사용한다.

$$V = \begin{bmatrix} 0.1 & 0.2 & -0.5 \\ 0.3 & 0.1 & -0.3 \end{bmatrix}$$

$$W = \begin{bmatrix} 0.1 & 0.2 \end{bmatrix}$$



입력 패턴	목표치
A	-1 (클러스터 1)
B	+1 (클러스터 2)
C	+1 (클러스터 2)
D	-1 (클러스터 1)

$$NET_z = XV^T = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0.1 & 0.3 \\ 0.2 & 0.1 \\ -0.5 & -0.3 \end{bmatrix} = \begin{bmatrix} 0.1 & 0.3 \end{bmatrix}$$

$$Z = \frac{1}{1 + e^{-NET_z}} = \begin{bmatrix} 0.525 & 0.574 \end{bmatrix}$$

$$NET_y = ZW^T = \begin{bmatrix} 0.525 & 0.574 \end{bmatrix} \begin{bmatrix} 0.1 \\ 0.2 \end{bmatrix} = 0.167$$

$$y = \frac{1}{1 + e^{-NET_y}} = 0.542$$

$$E = \frac{1}{2} (d - y)^2 = 0.5 \cdot (-1 - 0.542)^2 = 1.189$$

Training MLP

(Example)

예제 13.1과 동일한 조건에서 학습 패턴 A를 입력하여 신경망을 학습시키는 경우, 은닉층과 출력층간의 연결 강도 변화량 ΔW 를 구하라. 단, 학습률 α 는 1이다.

$$\delta_y = (d - y)y(1 - y) = (-1 - 0.542)0.542(1 - 0.542) = -0.383$$

$$\Delta W = \alpha \delta_y Z = 1 \cdot (-0.383) \cdot [0.525 \quad 0.574] = [-0.201 \quad -0.220]$$

예제 13.1과 동일한 조건에서 학습 패턴 A를 입력하여 신경망을 학습시키는 경우, 입력층과 은닉층간의 연결 강도 변화량 ΔV 를 구하라. 단, 학습률 α 는 1이다.

$$\delta_{z,1} = z_1(1 - y) \delta_y w_1 = 0.525 \cdot (1 - 0.542) \cdot (-0.383) \cdot 0.1 = -0.009$$

$$\delta_{z,2} = z_2(1 - y) \delta_y w_2 = 0.574 \cdot (1 - 0.542) \cdot (-0.383) \cdot 0.2 = -0.020$$

$$\Delta V = \alpha \delta_z X = 1 \cdot \begin{bmatrix} -0.009 \\ -0.020 \end{bmatrix} [1 \quad 0 \quad 0] = \begin{bmatrix} -0.009 & 0 & 0 \\ -0.020 & 0 & 0 \end{bmatrix}$$

Training MLP

- Training a 2-layer network => only needs 20 lines

```
1 import numpy as np
2 from numpy.random import randn
3
4 N, D_in, H, D_out = 64, 1000, 100, 10
5 x, y = randn(N, D_in), randn(N, D_out)
6 w1, w2 = randn(D_in, H), randn(H, D_out)
7
8 for t in range(2000):
9     h = 1 / (1 + np.exp(-x.dot(w1)))
10    y_pred = h.dot(w2)
11    loss = np.square(y_pred - y).sum()
12    print(t, loss)
13
14    grad_y_pred = 2.0 * (y_pred - y)
15    grad_w2 = h.T.dot(grad_y_pred)
16    grad_h = grad_y_pred.dot(w2.T)
17    grad_w1 = x.T.dot(grad_h * h * (1 - h))
18
19    w1 -= 1e-4 * grad_w1
20    w2 -= 1e-4 * grad_w2
```

Define the network

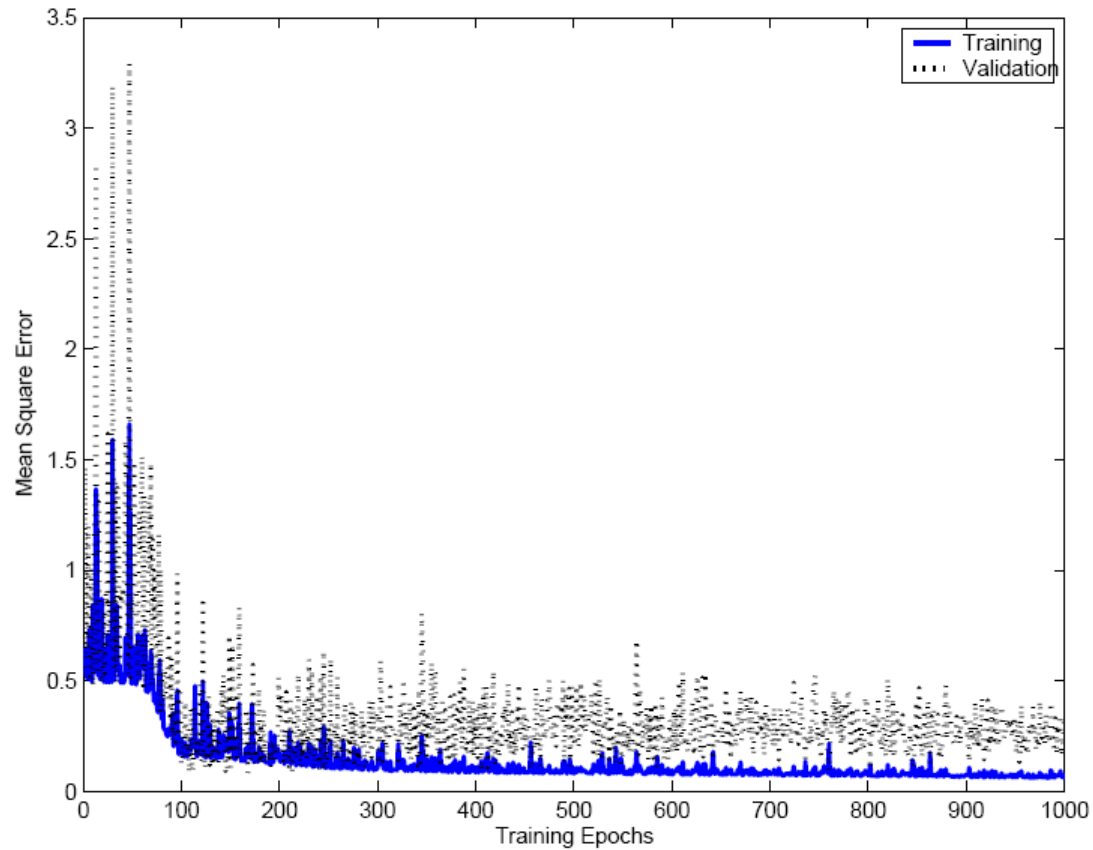
Forward pass

Calculate the analytical gradients

Gradient descent

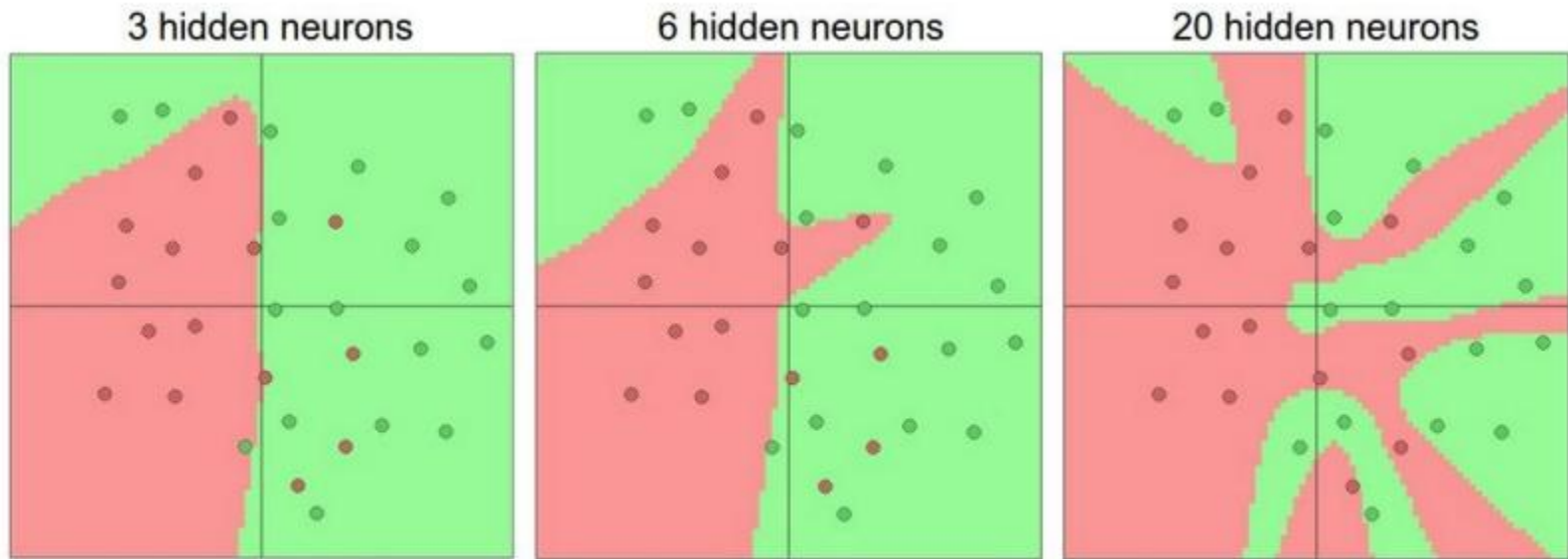
Training MLP

- Training Error / Validation Error



Training MLP

- Setting the number of layers



↑
more neurons = more capacity

Training MLP

- Hyperparameter

- 1) 초기 연결 강도 (weight)

- 연결강도는 국지적 최적값에 수렴
- 초기 연결강도에 따라 결과 달라짐
- -0.5 - 0.5

- 2) 신경망 구조

- 입력층 / 출력층 수: 문제에 따라 결정
- 은닉층 수: 적절히 결정하여야 함
 - 적은 경우 - 학습이 안 되는 현상 (에러가 줄어들지 않음)
 - 많은 경우 - 학습 속도가 느림, 에러가 줄어들지 않음

- 3) 학습률

- 큰 경우: 학습 속도 빠름, 학습 이루어지지 않을 수 있음 (overshoot)
- 작은 경우: 학습속도 느림

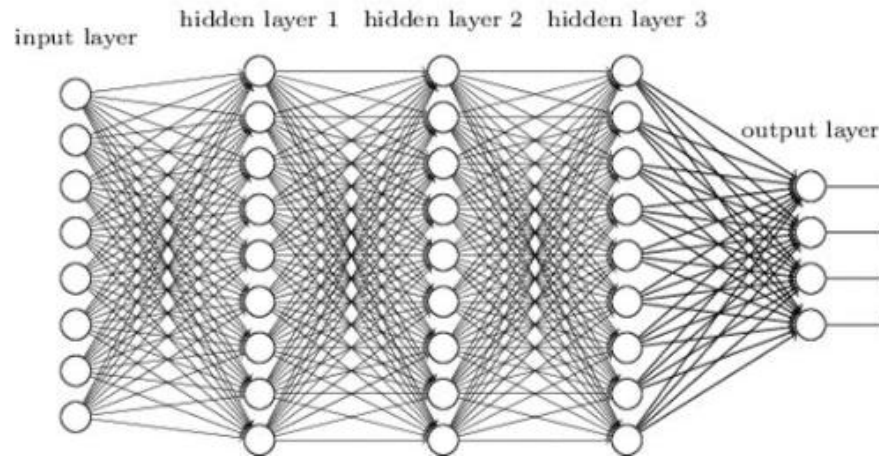
- 4) 모멘텀

- 학습 초반에는 변화량을 크게, 학습 후반에는 변화량을 작게 => 학습 속도 향상

$$\Delta V_k = \alpha \delta_x + \beta \Delta V_{k-1} \quad \beta \text{ 모멘텀상수}$$

$$\Delta W_k = \alpha \delta_y + \beta \Delta W_{k-1}$$

Deep Neural Network



- DNN 학습 (BP 알고리즘) 의 문제점
 - 학습 오차 문제 (Error)
 - 학습 속도 문제 (Time)
 - 과적합 문제 (Overfitting)
- 1990~2000년대에는 시험적 연구만 진행

Weight Update

- 신경망 학습이란?
 - Loss function 을 최소화하는 weight 를 찾는 문제 (optimization problem)
- 신경망 학습 방법
 - weight 초기값을 임의로 설정하고, weight 를 반복적으로 update (improvement)
 - Weight 변화량은 loss function 의 기울기 크기에 비례 (gradient descent)
 - 1) Stochastic Gradient Descent (SGD)
 - 2) Momentum
 - 3) AdaGrad (Adaptive Gradient)
 - 4) Adam

Stochastic Gradient Descent

- Algorithm

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

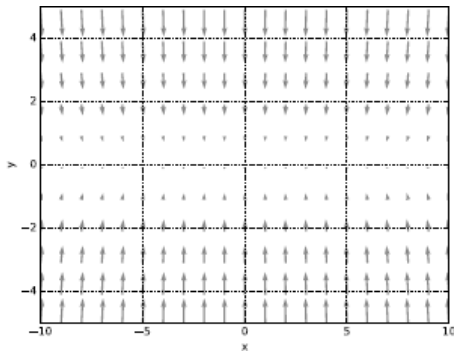
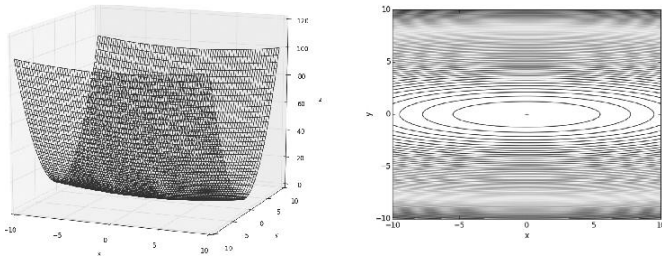
η : learning rate

```
class SGD:
    def __init__(self, lr=0.01):
        self.lr = lr

    def update(self, params, grads):
        for key in params.keys():
            params[key] -= self.lr * grads[key]
```

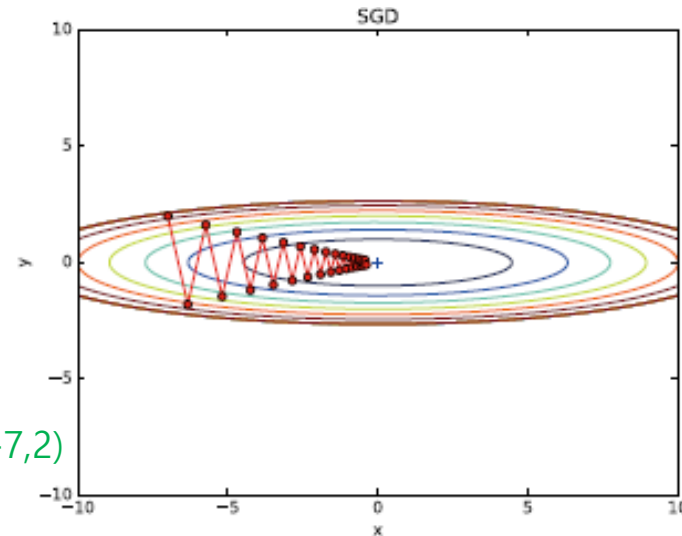
(예)

$$f(x, y) = \frac{1}{20}x^2 + y^2$$



기울기(gradient)

Y축 방향은 가파르고 x축 방향은 완만
=> 지그재그 이동



초기값 (-7,2)

단점: 비등방성 (anisotropy) 함수에 대한 탐색경로가 비효율적임

Momentum

- Algorithm

$$\begin{aligned} \mathbf{v} &\leftarrow \alpha \mathbf{v} - \eta \frac{\partial L}{\partial \mathbf{W}} \\ \mathbf{W} &\leftarrow \mathbf{W} + \mathbf{v} \end{aligned}$$

α : coefficient of momentum

$$\nu_1 = -G_1$$

$$\nu_2 = -0.9 * G_1 - G_2$$

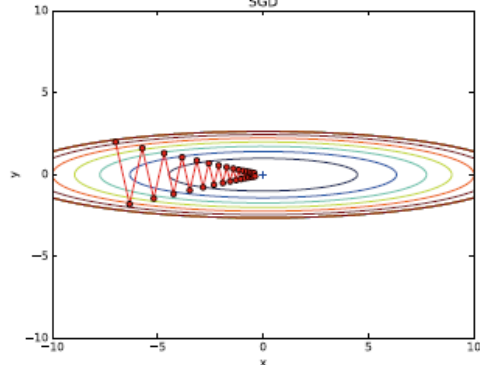
$$\nu_3 = -0.9 * (0.9 * G_1 - G_2) - G_3 = -0.81 * (G_1) - (0.9) * G_2 - G_3$$

- 효과

- 과거 업데이트 값도 현재 업데이트에 반영
- 관성효과로 빠른 이동 가능
- 지그재그 탐색 완화
- Local minima 를 벗어나는 효과 기대

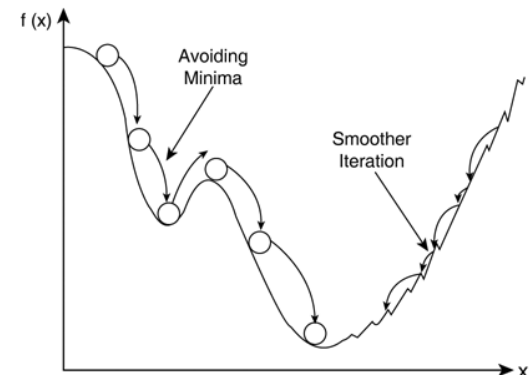
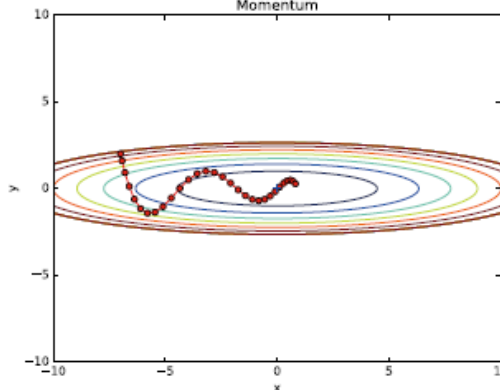
$\alpha = 0.0$

SGD



$\alpha = 0.9$

Momentum



AdaGrad

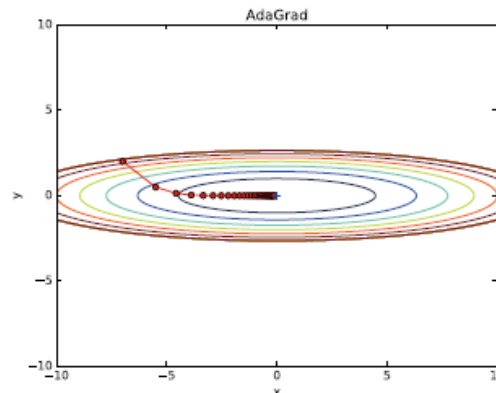
- Adaptive Gradient (2011)

- 학습을 진행하면서 learning rate (학습률) 을 점차 감소시키는 방법

$$\mathbf{h} \leftarrow \mathbf{h} + \frac{\partial L}{\partial \mathbf{W}} \odot \frac{\partial L}{\partial \mathbf{W}}$$
$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{1}{\sqrt{\mathbf{h}}} \frac{\partial L}{\partial \mathbf{W}}$$

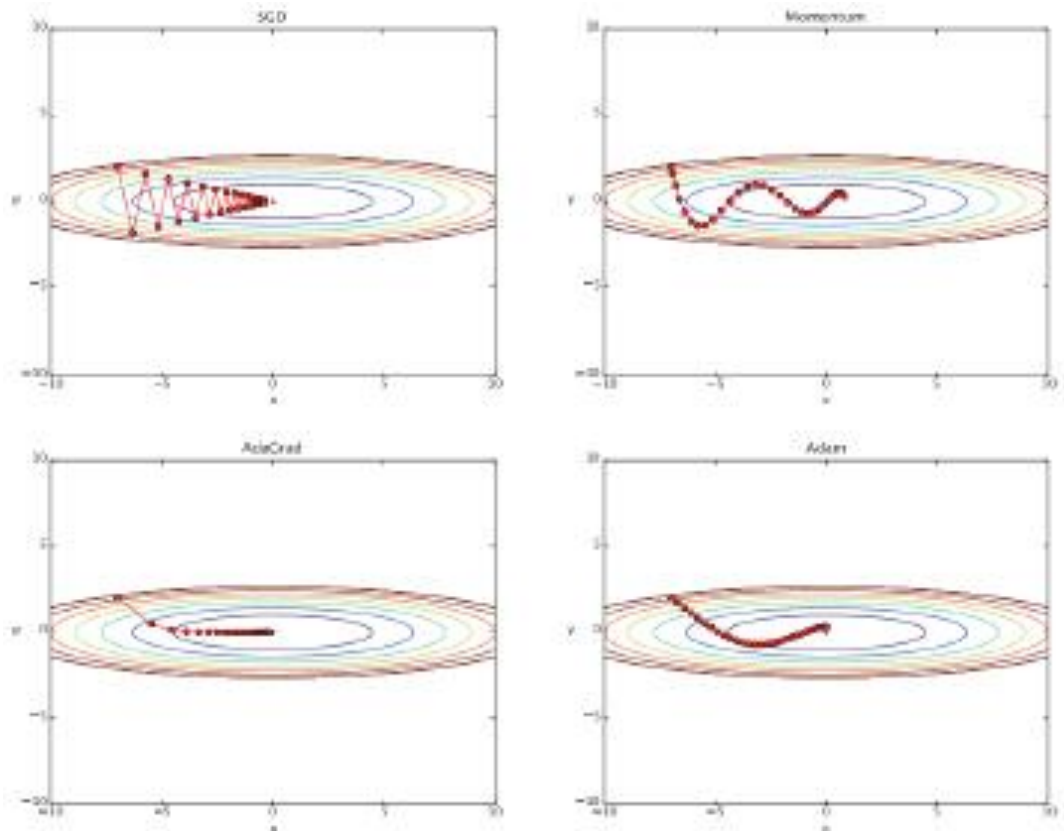
- 지금까지 적게 변화한 weight 변화량 (step size) 을 크게함 => 빠른 이동
- 지금까지 많이 변화한 weight 변화량 (step size) 을 작게함=> 미세 조정
- 학습이 오래 진행되면 step size 가 너무 줄어들어 update 중단

- RMSProp
- AdaDelta



Adam

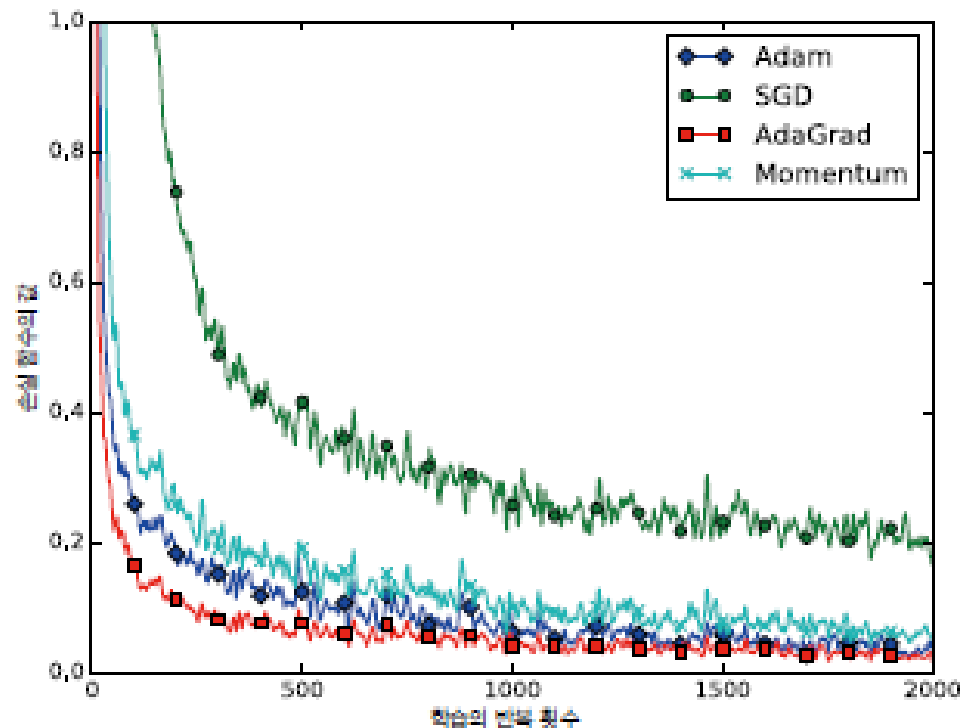
- Adaptive Momentum Estimation (2015)
 - Momentum + AdaGrad (RMSProp) 융합



Weight Update

- Optimization 방법 비교

그림 6-9 MNIST 데이터셋에 대한 학습 진도 비교



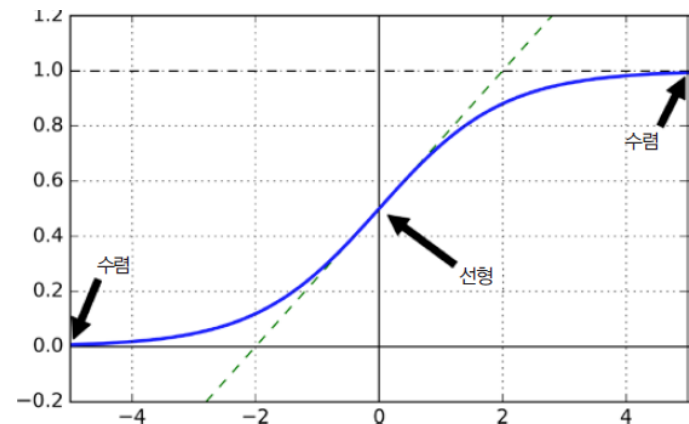
Gradient Trouble

- Gradient Vanishing (소실)
 - gradient 값이 점점 작아짐
 - Weight update 중단 => 학습 오차가 떨어지지 않음
- Gradient Exploding (폭주)
 - gradient 값이 점점 커짐
 - 비정상적으로 큰 값으로 Weight update => 발산

$$\begin{aligned}\Delta w_{ij} &= -\eta \frac{\partial E}{\partial w_{ij}} \\ &= -\eta \frac{\partial E}{\partial y} \frac{\partial y}{\partial z_j} \frac{\partial z_j}{\partial w_{ij}}\end{aligned}$$

Gradient Trouble

- 원인 (X.Glorot, Y.Bengio, 2010)
 - 1) Activation function: sigmoid function
 - 2) Weight initialization: 평균=0, 분산=1 정규분포
- 각 층에서 출력의 분산이 입력의 분산보다 큼
 - 상위 층으로 갈수록 활성화 함수값이 0 또는 1로 수렴
 - 기울기 (gradient) 가 0 에 가까워짐
 - 상위층 부터 역전파 진행 시 하위층에는 오류 전파 안됨
 - weight update stop
 - 학습오차가 줄어들지 않음



Gradient Trouble

- 해결 방법
 - 1) Activation Function: ReLU 및 그 변종
 - 2) Weight Initialization: Xavier, He 등
 - 3) Batch normalization

Activation Function

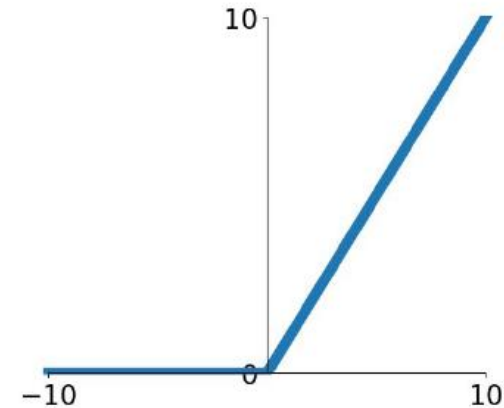
- ReLU

- Rectified Linear Unit

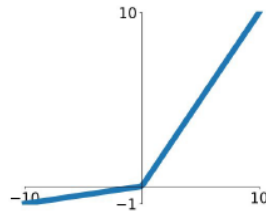
- $F(x) = \max(0, x)$

- 장점

- Saturation 이 없다
 - Very computationally efficient
 - Converges much faster than sigmoid/tanh

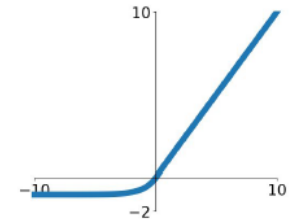


Leaky ReLU
 $\max(0.1x, x)$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Exponential linear unit

Weight Initialization

- 은닉층의 출력값 (활성화값) 분포 (histogram)

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

x = np.random.randn(1000, 100) # 1000개의 데이터
node_num = 100                # 각 은닉층의 노드(뉴런) 수
hidden_layer_size = 5         # 은닉층이 5개
activations = []              # 이곳에 활성화 결과(활성화값)를 저장

for i in range(hidden_layer_size):
    if i != 0:
        x = activations[i-1]

        w = np.random.randn(node_num, node_num) * 1
        a = np.dot(x, w)
        z = sigmoid(a)
        activations[i] = z

# 히스토그램 그리기
for i, a in activations.items():
    plt.subplot(1, len(activations), i+1)
    plt.title(str(i+1) + "-layer")
    plt.hist(a.flatten(), 30, range=(0,1))
plt.show()
```

- Input layer: 1000 nodes
- 5 hidden layers: 100 nodes / layer
- Activation function = sigmoid
- Weight 초기값: 평균 0 인 정규분포
분산 = 1 or 0.01

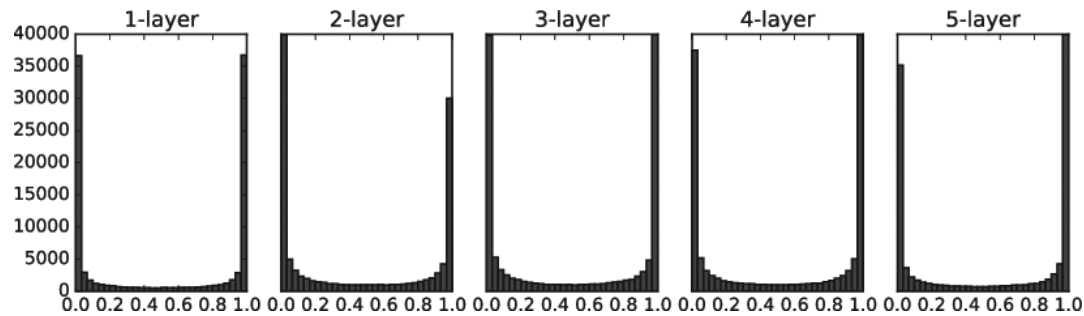


```
# w = np.random.randn(node_num, node_num) * 1
w = np.random.randn(node_num, node_num) * 0.01
```

Weight Initialization

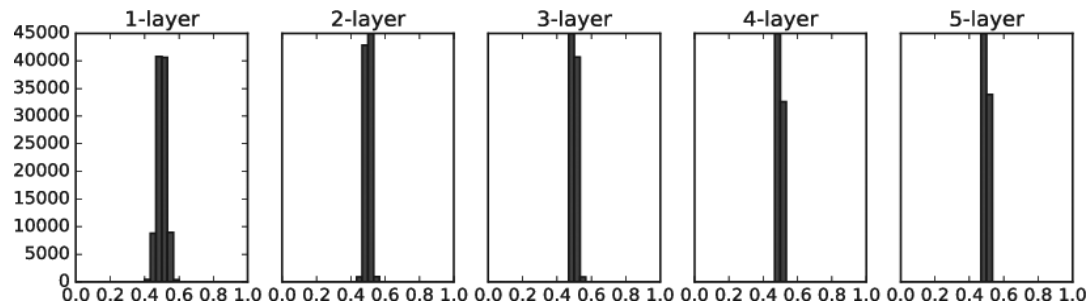
- Weight 를 정규분포로 초기화할 때의 은닉층의 출력값 분포 (histogram)

표준편차=1



0 또는 1 에 집중
→ 기울기 소실

표준편차=0.01

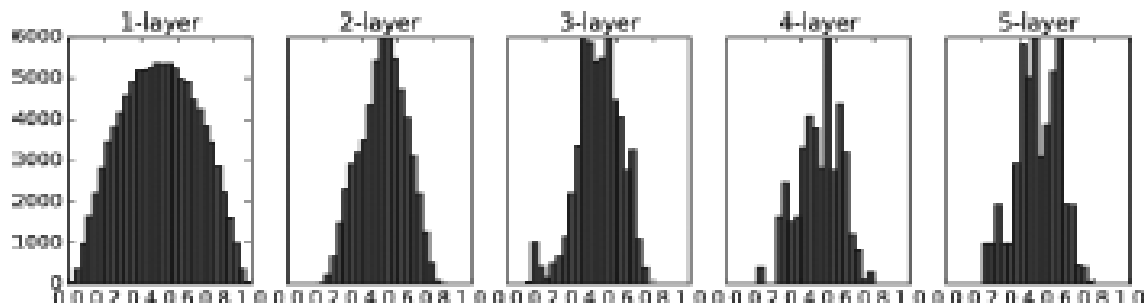
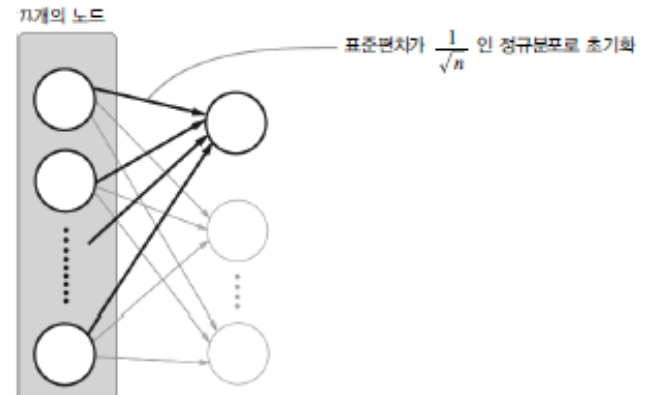


0.5 에 집중
→ 모든 neuron 이 유사값 출력
→ 표현력 제한

Weight Initialization

- Xavier (Glorot) 초기화 (2010)
 - Activation 함수: Sigmoid, tanh function
 - $\sigma = \frac{1}{\sqrt{n}}$, $n = fan_{in}$ = 앞층의 노드 수

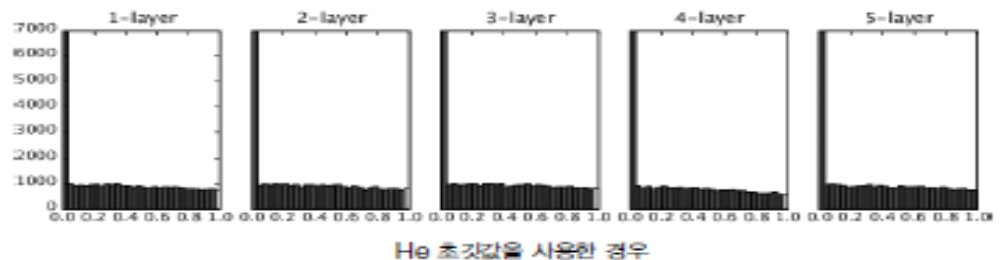
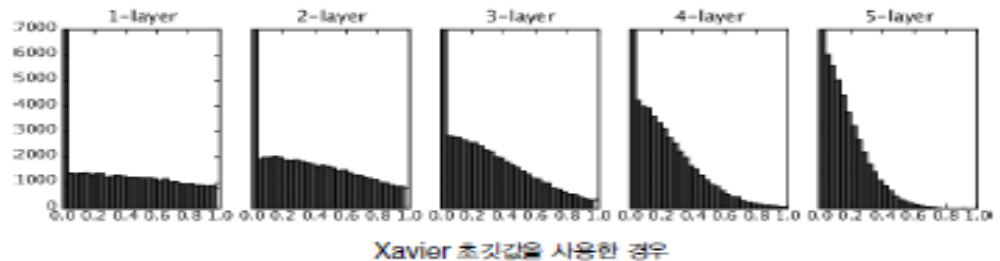
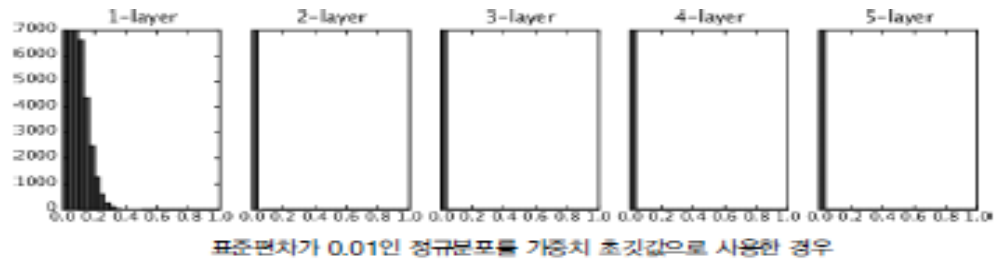
```
node_num = 100 # 앞 층의 노드 수  
w = np.random.randn(node_num, node_num) / np.sqrt(node_num)
```



Weight Initialization

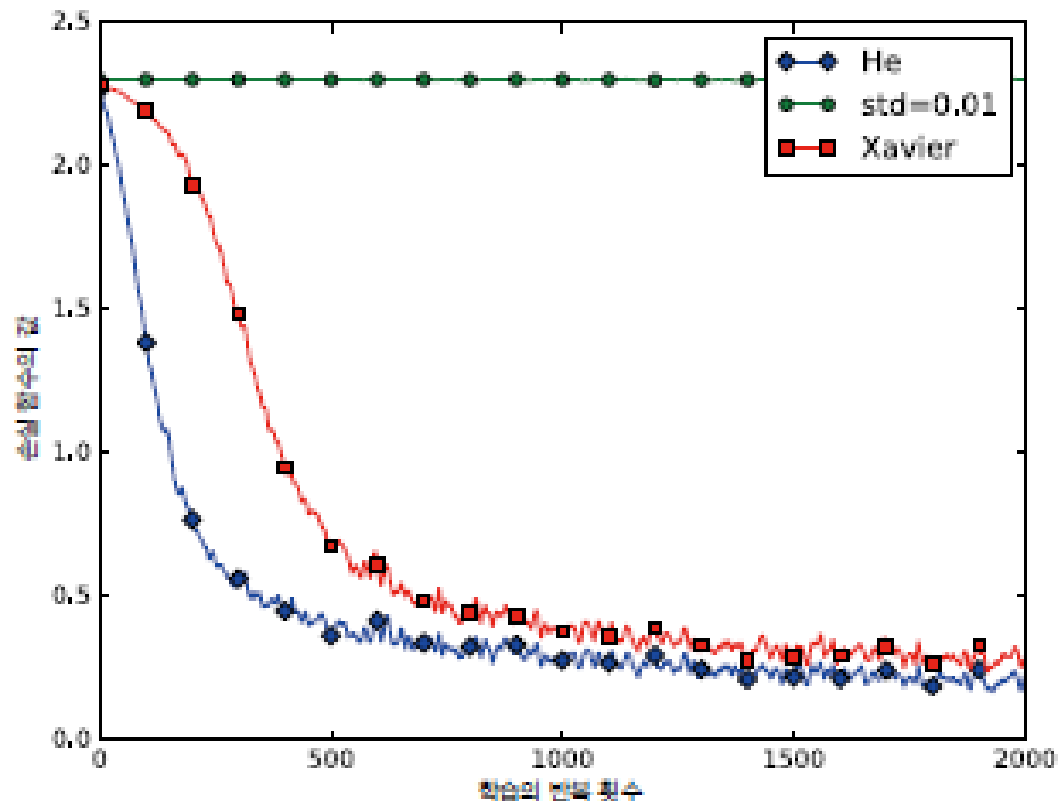
- He 초기화 (2015)
 - Activation function : ReLU 및 변종 들
 - $\sigma = \frac{2}{\sqrt{n}}$, $n = fan_{in}$ = 앞층의 노드 수

ReLU 사용 시 weight 초기값에 따른
은닉층 출력값 분포



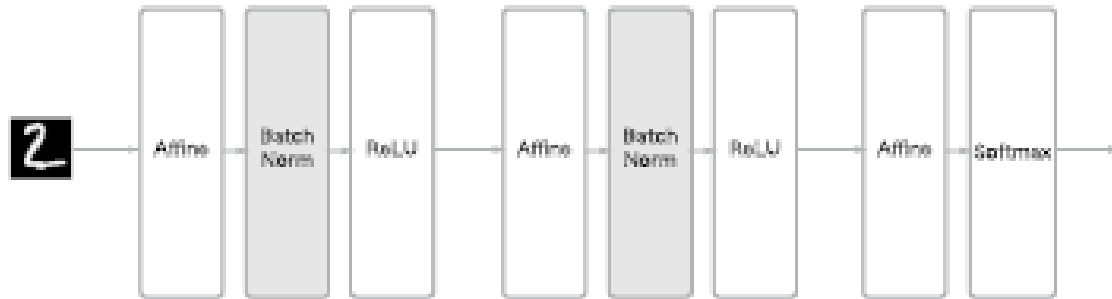
Weight Initialization

- MNIST 데이터셋 적용
 - ReLU function + He initialization



Batch Normalization

- Batch Normalization (S.Loffe & C.Szegedy, 2015)
 - Gradient vanishing/Exploding 방지 목적
 - Activation 함수 통과 전 또는 후 데이터 '정규화' 과정 추가



- Mini-batch 단위로 정규화: 평균 = 0, 분산 = 1

$$\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i$$

$$\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

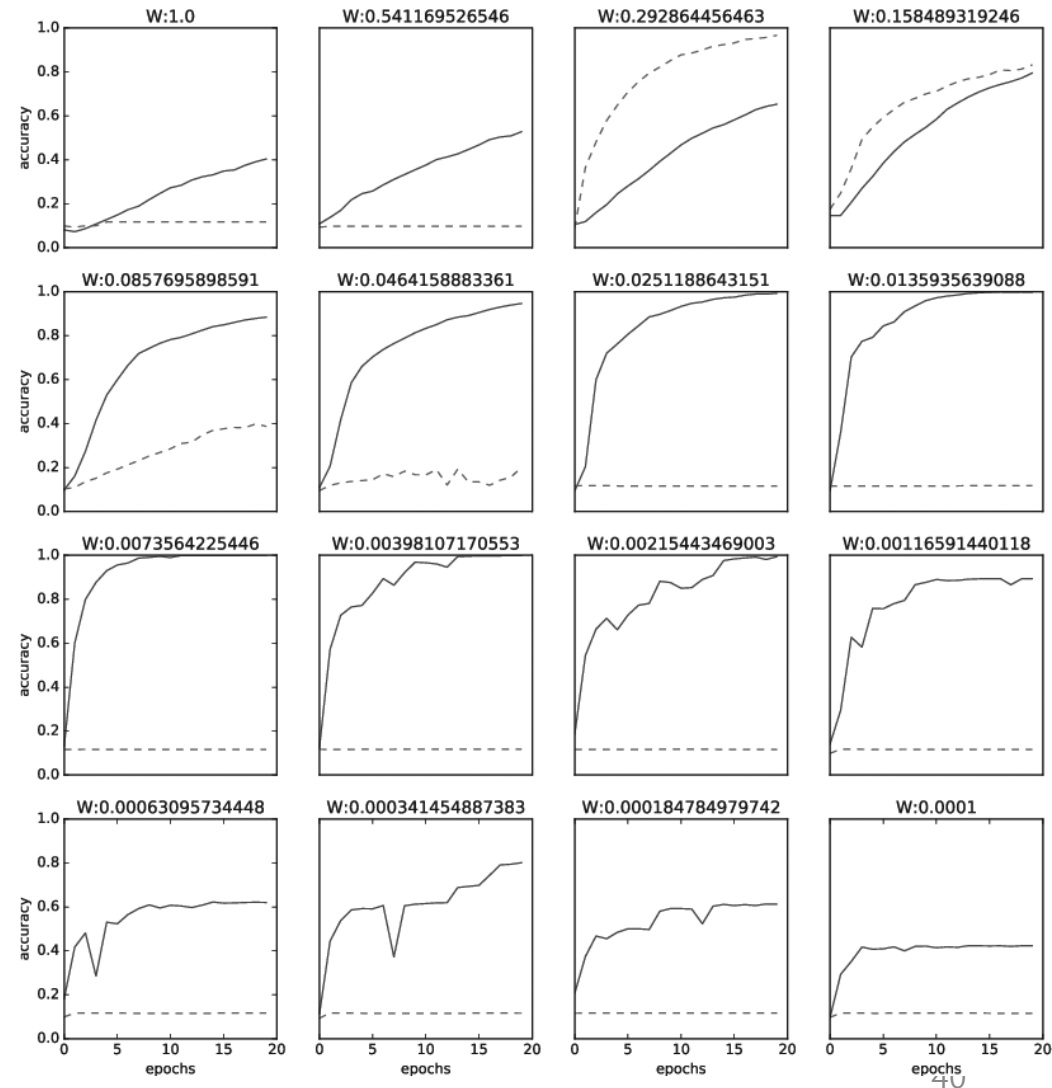
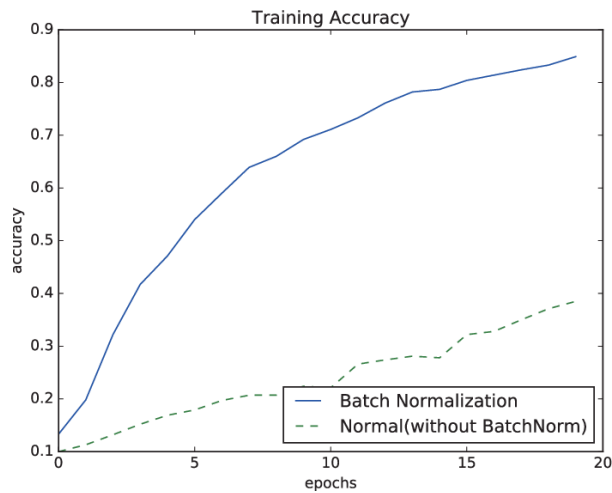
$$y_i \leftarrow \gamma \hat{x}_i + \beta$$

γ : scaling, β : shift

Batch Normalization

- 효과

- 학습속도 향상
- Weight 초기값 영향이 적음



Overfitting

- Overfitting (과적합)

- 신경망이 훈련 데이터에 지나치게 적응. 그 외의 데이터에는 대응하지 못하는 상태

- 원인

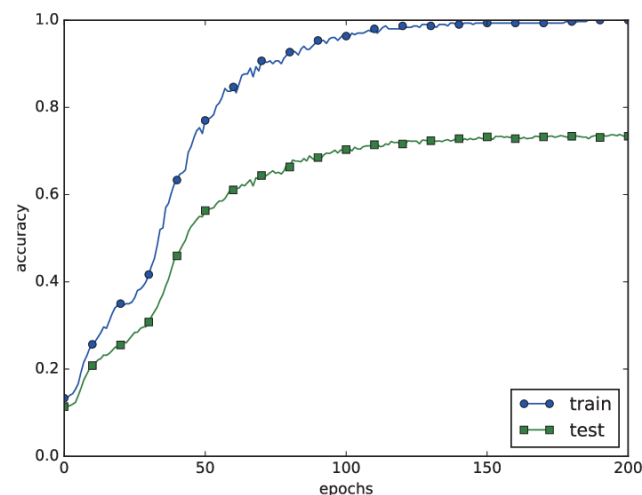
- 매개변수가 많고 표현력이 높은 모델
 - 훈련데이터가 적음

- 예

- 7 layer network
 - 100 neurons/layer, ReLU
 - MNIST 데이터셋 (60,000개)
 - training: 300개

- 해결방안

- Weight decay
 - Dropout



Weight Decay

- Weight Decay (가중치 감소)

- Weight 값이 커지는 것을 억제 => overfitting 억제 효과
- Loss function 에 weight norm 추가

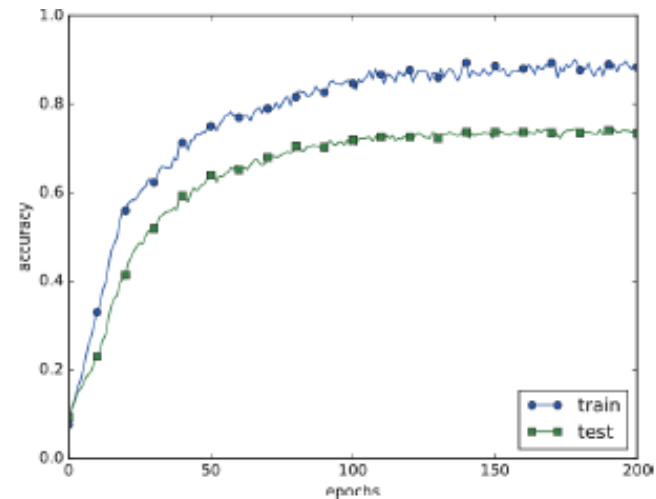
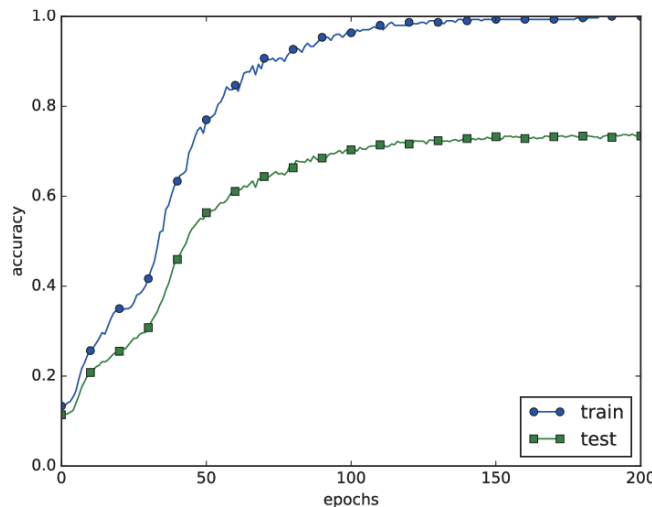
$$E = \frac{1}{2} \sum_k (y_k - t_k)^2 + \frac{1}{2} \lambda W^2$$

$$W = (w_1, w_2, \dots, w_n)$$

L2 norm : $\sqrt{w_1^2 + w_2^2 + \dots + w_n^2}$

L1 norm : $|w_1| + |w_2| + \dots + |w_n|$

L ∞ norm : $\max(|w_1|, |w_2|, \dots, |w_n|)$



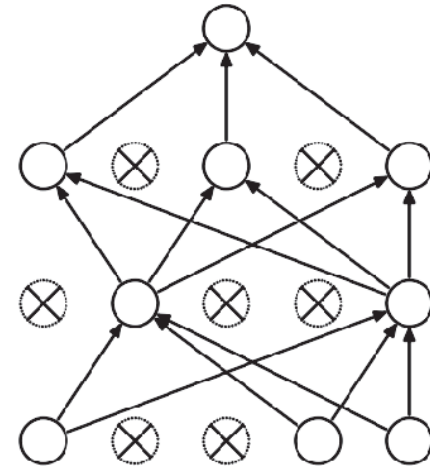
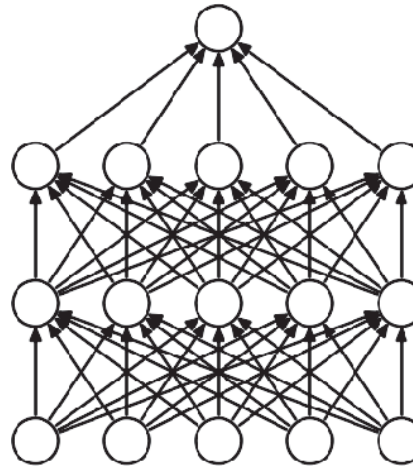
Dropout

- Dropout
 - Training 시
 - Dropout 비율 (dropout_ratio) 에 해당하는 neuron 은 학습에서 삭제
 - Test 시
 - 각 neuron 의 출력에 training 시 삭제 안 한 비율을 곱하여 출력

```
class Dropout:
    def __init__(self, dropout_ratio=0.5):
        self.dropout_ratio = dropout_ratio
        self.mask = None

    def forward(self, x, train_flg=True):
        if train_flg:
            self.mask = np.random.rand(*x.shape) > self.dropout_ratio
            return x * self.mask
        else:
            return x * (1.0 - self.dropout_ratio)

    def backward(self, dout):
        return dout * self.mask
```



Dropout

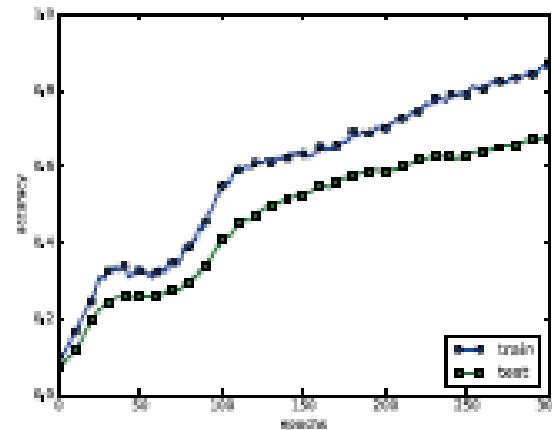
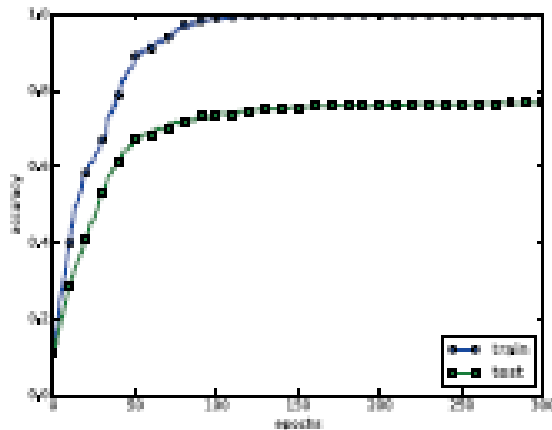
- Dropout 의 효과

- Ensemble learning 효과

- 개별적으로 학습시킨 여러 개의 모델 출력 활용

- Tabu search 효과

- Hill-climbing (Gradient descent) 에 의한 weight 탐색 개선 => 탐색 공간 다변화



dropout_ratio=0.15

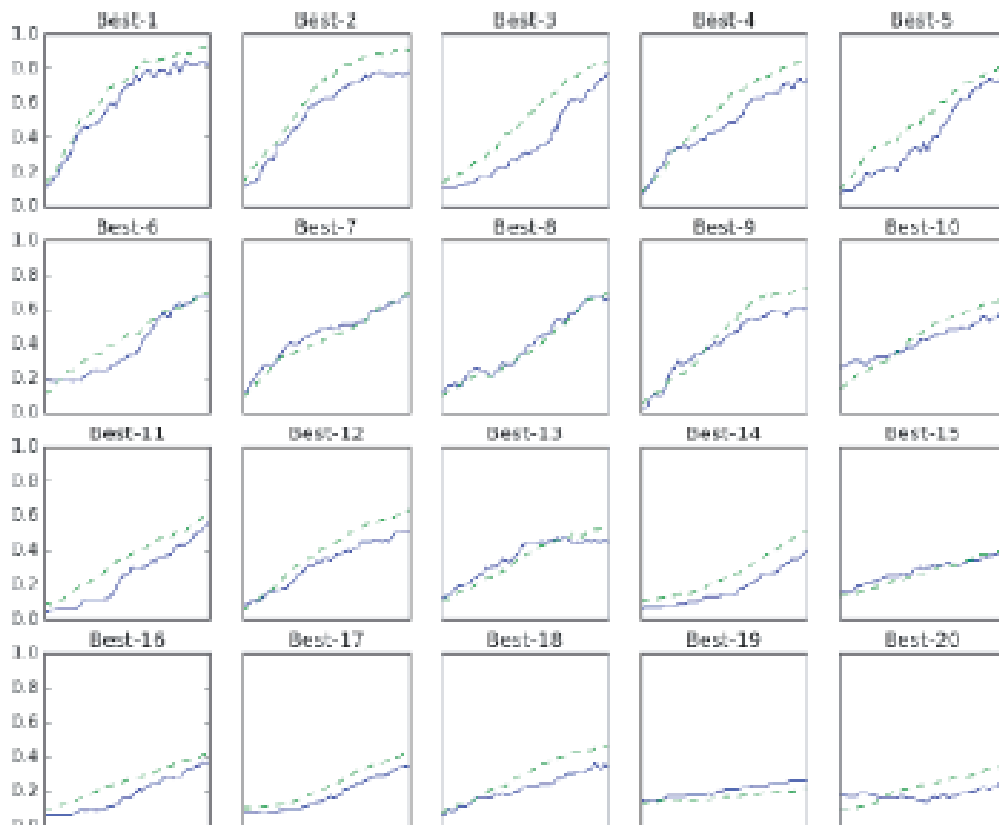
Hyperparameter Optimization

- Hyperparameter
 - Layer 수, neuron 수, batch size, ...
 - Learning rate, momentum coefficient, ...
 - Weight decay, dropout ratio, ...
- Training data vs. validation data vs. test data
 - Training data: weight 학습용
 - Validation data: hyperparameter 검증용
 - Test data: 성능평가용

Hyperparameter Optimization

- Example

그림 6-24 실선은 검증 데이터에 대한 정확도, 점선은 훈련 데이터에 대한 정확도



```
weight_decay = 10 ** np.random.uniform(-8, -4)  
lr = 10 ** np.random.uniform(-6, -2)
```

Best-1 (val acc:0.83) | lr:0.0092, weight decay:3.86e-07
Best-2 (val acc:0.78) | lr:0.00956, weight decay:6.04e-07
Best-3 (val acc:0.77) | lr:0.00571, weight decay:1.27e-06
Best-4 (val acc:0.74) | lr:0.00626, weight decay:1.43e-05
Best-5 (val acc:0.73) | lr:0.0052, weight decay:8.97e-06